

Prompt: complete implementation blueprint for **US-04 Resume Upload**, covering both the backend Node.js (Express) logic and the frontend React components.

⚙️ Backend Implementation (Node.js & Express)

This code uses **multer** to handle the multipart/form-data upload, enforces the **strictly PDF** and **5MB size limits**, saves the file to a local uploads directory, and updates the **MySQL database**.

```
const express = require('express');
const multer = require('multer');
const path = require('path');
const mysql = require('mysql2/promise');

const app = express();

// 1. Database Connection Setup
const db = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'careerconnect'
});

// 2. Configure Storage Engine & Unique Filenames
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/resumes/');
  },
  filename: (req, file, cb) => {
    // Appends timestamp to prevent overwriting files with
    // identical names
    const uniqueSuffix = Date.now() + '-' +
      Math.round(Math.random() * 1E9);
    cb(null, `${uniqueSuffix}-${file.originalname}`);
  }
});

// 3. File Filter Strategy (Strictly PDF Enforced)
const fileFilter = (req, file, cb) => {
  if (file.mimetype === 'application/pdf') {
    cb(null, true);
  } else {
    cb(new Error('Invalid file type. Only PDF files are
```

```

allowed!'), false);
    }
};

const upload = multer({
  storage: storage,
  limits: { fileSize: 5 * 1024 * 1024 }, // 5MB Limit Enforced
  fileFilter: fileFilter
});

// 4. POST Endpoint: Handle Resume Upload
app.post('/api/resumes/upload', upload.single('resume'), async
(req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({ error: 'Please upload a
file.' });
    }

    // Mock User ID (In production, retrieve this safely from
your JWT/session auth)
    const userId = 1;
    const { originalname, path: filePath, size } = req.file;

    // Insert metadata record into MySQL
    const query = 'INSERT INTO resumes (user_id, file_name,
file_path, file_size_bytes) VALUES (?, ?, ?, ?)';
    await db.execute(query, [userId, originalname, filePath,
size]);

    res.status(201).json({
      message: 'Resume uploaded successfully!',
      fileName: originalname
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
});

// Global error handling middleware for handling Multer file size
exceptions
app.use((err, req, res, next) => {
  if (err instanceof multer.MulterError && err.code ===
'LIMIT_FILE_SIZE') {
    return res.status(400).json({ error: 'File size is too
large. Maximum allowed size is 5MB.' });
  }
});

```

```

    if (err) {
      return res.status(400).json({ error: err.message });
    }
    next();
  });
}

app.listen(5000, () => console.log('Server running on port 5000'));

```

Frontend Implementation (React & Tailwind CSS)

This component builds a user-friendly drag-and-drop file upload zone. It includes real-time **loading spinner feedback**, **success/error validation alerts**, and switches dynamically to a **completed state** layout showing the file name.

```

import React, { useState } from 'react';

export default function ResumeUpload() {
  const [file, setFile] = useState(null);
  const [loading, setLoading] = useState(false);
  const [message, setMessage] = useState({ text: '', isError: false });

  // Handle local file picker selections
  const handleFileChange = (e) => {
    const selectedFile = e.target.files[0];
    validateAndSetFile(selectedFile);
  };

  // Enforce frontend checks before bothering the server
  const validateAndSetFile = (selectedFile) => {
    if (!selectedFile) return;

    if (selectedFile.type !== 'application/pdf') {
      setMessage({ text: 'Only PDF documents are supported.',
isError: true });
      return;
    }

    if (selectedFile.size > 5 * 1024 * 1024) {
      setMessage({ text: 'File exceeds the 5MB size limit.',
isError: true });
      return;
    }
  };
}

```

```

    }

    setFile(selectedFile);
    setMessage({ text: '', isError: false });
  };

  // Async API submission block
  const handleUploadSubmit = async (e) => {
    e.preventDefault();
    if (!file) return;

    setLoading(true);
    setMessage({ text: '', isError: false });

    const formData = new FormData();
    formData.append('resume', file);

    try {
      const response = await fetch('/api/resumes/upload', {
        method: 'POST',
        body: formData,
      });

      const data = await response.json();

      if (!response.ok) throw new Error(data.error || 'Upload
failed');

      setMessage({ text: 'Resume uploaded and saved successfully!',
isError: false });
    } catch (error) {
      setMessage({ text: error.message, isError: true });
      setFile(null); // Reset file context on failure
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="max-w-md mx-auto mt-10 p-6 bg-white rounded-xl
shadow-md border border-gray-200">
      <h2 className="text-xl font-bold text-gray-800 mb-4">Resume
Document</h2>

      <form onSubmit={handleUploadSubmit} className="space-y-4">
        {!file ? (
          // Empty Drop Zone State
          <div className="border-2 border-dashed border-gray-300

```

```

rounded-lg p-6 text-center hover:border-blue-500 transition
cursor-pointer relative">
    <input
      type="file"
      accept=".pdf"
      onChange={handleFileChange}
      className="absolute inset-0 w-full h-full opacity-0
cursor-pointer"
    />
    <p className="text-gray-600 font-medium">Click to
browse or drag your resume here</p>
    <p className="text-xs text-gray-400 mt-1">Accepts PDF
files up to 5MB</p>
  </div>
) : (
  // Completed Selection State
  <div className="flex items-center justify-between p-4
bg-blue-50 border border-blue-200 rounded-lg">
    <div className="flex items-center space-x-3">
      <span className="text-2xl"><img alt="document icon" data-bbox="585 415 605 435"/></span>
      <span className="text-sm font-medium text-blue-800
truncate max-w-[200px]">{file.name}</span>
    </div>
    <button
      type="button"
      onClick={() => setFile(null)}
      className="text-sm font-semibold text-red-600
hover:text-red-800"
      disabled={loading}
    >
      Remove
    </button>
  </div>
) }

{/* Global Notifications Panel */}
{message.text && (
  <div className={`p-3 rounded-md text-sm font-medium
${message.isError ? 'bg-red-50 text-red-700' : 'bg-green-50
text-green-700'}`}>
    {message.text}
  </div>
) }

{/* Submit Action Button */}
{file && !message.text && (
  <button
    type="submit"

```

```

        disabled={loading}
        className="w-full bg-blue-600 hover:bg-blue-700
text-white font-semibold py-2 px-4 rounded-lg shadow-sm transition
disabled:opacity-50 flex justify-center items-center"
      >
        {loading ? (
          <span className="animate-spin inline-block w-5 h-5
border-2 border-white border-t-transparent rounded-full
mr-2"></span>
        ) : 'Upload Document'}
      </button>
    )}
  </form>
</div>
);
}

```

Would you like me to show you how to expand this foundational system to handle **Story 5 (Multiple Resume Management)** so users can **label and store variations** of their profiles?